

On the Effectiveness of Mempool-based Transaction Auditing

Jannik Albrecht
jannik.albrecht@ruhr-uni-bochum.de
Ruhr University Bochum
Bochum, Germany

Ghassan Karame
ghassan@karame.org
Ruhr University Bochum
Bochum, Germany

Abstract

While the literature features a number of proposals to defend against transaction manipulation attacks, existing proposals are still not integrated within large blockchains, such as Bitcoin, Ethereum, and Cardano. Instead, the user community opted to rely on more practical but ad-hoc solutions (such as Mempool.space) that aim at *detecting* censorship and transaction displacement attacks by auditing discrepancies in the mempools of so-called observers.

In this paper, we precisely analyze, for the first time, the interplay between mempool auditing and the ability to detect censorship and transaction displacement attacks by malicious miners in Bitcoin and Ethereum. Our analysis shows that mempool auditing can result in mis-accusations against miners with a probability larger than 25% in some settings. On a positive note, however, we show that mempool auditing schemes can successfully audit the execution of any two transactions (with an overwhelming probability of 99.9%) if they are consistently received by all observers and sent at least 30 seconds apart from each other. As a direct consequence, our findings show, for the first time, that batch-order fair-ordering schemes can offer only strong fairness guarantees for a limited subset of transactions in real-world deployments.

CCS Concepts

• **Networks** → **Network measurement**; • **Security and privacy** → **Distributed systems security**.

Keywords

Mempool-based Auditing, Distributed Measurements

ACM Reference Format:

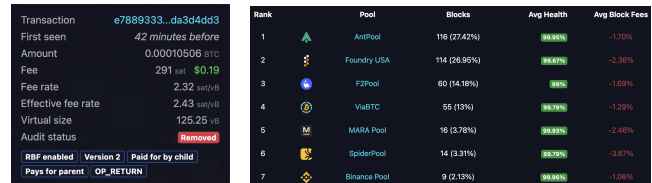
Jannik Albrecht and Ghassan Karame. 2026. On the Effectiveness of Mempool-based Transaction Auditing. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792404>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18336721>.

1 Introduction

Decentralized platforms continue to gain considerable traction as they eliminate the need for intermediaries. Here, transactions are first propagated in the P2P layer across blockchain nodes; correct and verified transactions are buffered by these nodes in a local



(a) A potentially censored transaction. (b) Block health ranking of the leading Bitcoin pools.

Figure 1: An excerpt of the block audit by the Mempool Open Source Project [3]. Figure (a) shows an audit of a block's removed, i.e., potentially censored, transaction. Figure (b) shows Mempool.space's ranking of the leading Bitcoin pools in terms of block health.

data structure, the *mempool*. Transactions in the mempool are then confirmed by miners—specific nodes that are rewarded for their effort—into so-called blocks. While this establishes a clear temporal order among transactions confirmed in different blocks, a malicious miner can censor, displace, or re-order unconfirmed transactions in its local mempool to increase its advantage in the system (e.g., by prioritizing transactions that pay higher fees or by dropping or censoring transactions). Given that these attacks exploit intrinsic properties of existing public blockchains, finding practical clean-slate solutions against them emerges as a significant research challenge that received considerable attention from both industry and academia [15, 17, 27, 31]. Existing remedies consist of preventing miners from enforcing a particular order when confirming blocks [23, 24, 26]. Unfortunately, these approaches either require considerable changes to the underlying blockchain protocols [26] or *require consistent transaction ordering—an assumption that has not been empirically evaluated—to provide so-called batch-fair ordering*. Due to these limitations, such solutions are still not integrated into large blockchains, such as Bitcoin, Ethereum, Cardano, etc.

Instead, the community opted to rely on practical but ad-hoc solutions [1–3, 9, 11, 14, 32, 33] that aim at *detecting* censorship and transaction displacement attacks by untrusted miners by observing discrepancies in the mempools of so-called observer nodes. More precisely, these solutions aim to identify whether the transactions received by independent network observers match those confirmed in blocks output by the miners. We dub this approach *mempool-based auditing*. For instance, Mempool.space [3] and Litecoin.space [32] compute the so-called “block health” output by each mining pool: this metric measures the difference between the expected transactions in a block and the actual block output by a given mining pool. This can strongly influence users and deter malicious miners, as a few mining pools control mining/leader election in many popular cryptocurrencies [10, 20] (see Figure 1b).

In this paper, we set forth to precisely analyze, for the first time, the provisions of mempool auditing in detecting censorship and transaction displacement attacks by miners. Our analysis explores the precise guarantees that such *mempool-based* detection



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792404>

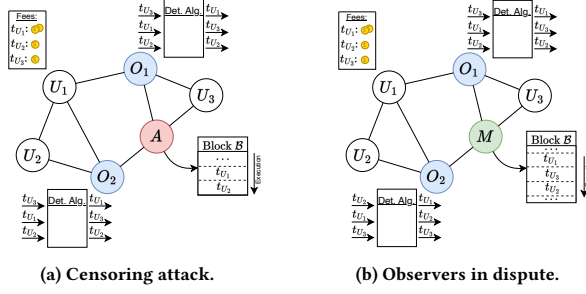


Figure 2: Examples showing adversary A executing displacement attacks: In Figure (a), adversary A censors transaction t_{U_3} by omitting it from block B , while two observers O_1 and O_2 expect t_{U_3} to be scheduled before t_{U_2} . Figure (b) shows a benign example, where both observers are in dispute on the rightful order of t_{U_2} and t_{U_3} .

approaches can give in realistic deployments, such as the current Bitcoin and Ethereum ecosystems. More precisely, we are interested in answering fundamental questions such as: *what is the precise probability that any two transactions sent in order will be observed in the same order by up to n independent observers? and what are the implications on the efficiency of batch-fair ordering schemes, Under what specific conditions can we confidently conclude that no censoring or transaction displacement attack has occurred?* This is particularly challenging to model since the local views of honest observers depend on factors such as connectivity, geographical distribution, and other network peculiarities, which drastically affect the guarantees exhibited by such auditing approaches. More specifically, we make the following contributions in this work:¹

Network model: We devise a realistic network model to demarcate the limits of (im-)possibility, and to capture all the relevant notions for mempool-based detection approaches. (Section 3).

Large-scale measurements: We validate our analytical model using large-scale measurements in Bitcoin and Ethereum. The combined outcomes of our formal and empirical analyses provide fundamental insights toward understanding the impact of the network layer on the ability to detect censorship and displacement attacks in the wild.

Effectiveness of mempool-auditing: Using both analytical and empirical methods, we show that mempool auditing exhibits limited generalizability and can lead to serious misaccusations due to inherent inconsistencies. More specifically, our findings reveal that a transaction that is sent more than 30 seconds earlier than another one is (almost) guaranteed to be received before the latter (cf. Section 3). Our results further suggest that mempool auditing can mislabel honest miners as “malicious” with a probability larger than 25% in some settings (cf. Section 4.2).

First analysis of batch-fair ordering schemes: Our results also have direct implications for evaluating the effectiveness of fair-ordering schemes. Specifically, we demonstrate that the key assumption underlying batch-fair ordering schemes—namely, that validators receive a consistent transaction order—is rarely satisfied in practice (cf. Section 5).

2 Problem Statement

While blocks impose a strict order on confirmed transactions, unconfirmed ones can be reordered by miners to gain advantage—for example, by prioritizing higher-fee transactions. Such manipulations are commonly referred to as front-running attacks, where adversaries observe pending transactions and attempt to have their own confirmed first. Miners may further delay transactions indefinitely, enabling censorship or targeted exclusion. These attacks are prevalent in practice, with consequences ranging from financial losses (e.g., arbitrage or stolen secrets) to censorship. For instance, Qin *et al.* [30] report losses of \$540.54M over 32 months (Dec 2018–Aug 2021) due to arbitrage attacks. In the sequel, we refer to all such manipulations as “transaction displacement attacks”².

Unfortunately, existing solutions to deter displacement attacks are impractical for existing blockchains. Namely, they either cannot support real-time detection/prevention or are not generically applicable to different blockchains/applications (see Section A in the Appendix for additional details). This gave rise to ad-hoc solutions [1–3, 9, 11, 14, 32, 33] that aim at *detecting* censorship and transaction displacement attacks by untrusted miners by observing discrepancies in the mempools of so-called observer nodes.

System Model. We consider a blockchain platform built atop a peer-to-peer (P2P) network connecting N nodes. Transactions are propagated among those nodes over the P2P layer. Each node stores the valid transactions it receives in its local mempool, a local data structure that maintains an ordered list of valid transactions. We assume that all honest nodes order transactions in their mempool based on their order of arrival. In practice, however, rational nodes tend to prioritize transactions that yield larger rewards, i.e., larger fees per weight ratio, to incorporate those in a block. Hence, we capture this strategy of rational miners³ by assuming the usage of a public deterministic algorithm (cf. Det. Alg. in Figure 2) to schedule the transactions’ execution based on their mempool order, their fees, their amounts, etc. This aligns with the behavior of widely used blockchain clients, such as bitcoind [7] and geth [18], which order transactions by reception time and then apply public and deterministic algorithms, such as fee-based sorting. Similar to [30, 34, 37], we assume that any deviation from this common strategy—such as the use of probabilistic or proprietary ordering—is an instance of censorship or misbehavior.

Given that the algorithm is public, observing either the input, e.g., the reception order, or the output, i.e., the sorted mempool, provides equivalent information for our analysis. In other words, given some input to the algorithm, the transactions’ order in mempool can be easily predicted for *honest nodes*. As soon as enough transactions have been received to fill a block, all transactions received at a later time are scheduled by miners to be confirmed in a subsequent block. We denote this time when enough transactions have been received to fill a block as τ_C (cf. Section 3.4).

²Terminology in the literature is inconsistent: “front-running” may denote both the broader class of order-manipulation attacks and the specific insertion of malicious transactions.

³Note that while mechanisms like Ethereum’s proposer-builder separation (PBS) divide block generation between multiple parties, such as proposers and builders, entities that are responsible for assembling blocks using their own local transaction views remain vulnerable to ordering inconsistencies.

¹The code and data required to reproduce our results are available at [22].

We assume that rational miners aim to increase their profit in the system while avoiding detection (i.e., to ensure that their reputation is not damaged (cf. Figure 1b)). *Specifically, we focus on the ability of malicious miners to select transactions from their local mempool and freely order them* to increase their advantage in the system. We do not consider the (similar) impact that non-miners can achieve by rushing their transactions in the system; indeed, while the impact of such attacks is detrimental, miners eventually have the ultimate decision in determining the eventual order of transactions and could correct this behavior.

Mempool-auditing. Several solutions in the community, such as mempool.space [3] and Etherscan [2], and some fair-ordering schemes, like Themis [24], rely on the core assumption that, since all nodes use a deterministic algorithm to sort their local mempool, the various validators should exhibit a similar order of transactions (assuming ideal network functionality). This core assumption allows for deterrence of misbehavior by *detecting* malicious miners who report a different transaction order in their confirmed blocks than the one witnessed by a “trusted” set of nodes, called observers, which are commissioned by a service provider to audit block/transaction ordering. An overview of existing Bitcoin and Ethereum blockchain detection tools is summarized in Table 5. These tools mostly monitor the reception of transactions and blocks and provide some web interface that logs pending transactions and their reception times (cf. [1, 2, 9, 11, 14, 33]). These logs can be utilized to (1) validate the reception of a transaction and (2) estimate the time for a transaction to be confirmed in a block. Unexpected deviations from the presumed confirmation schedule indicate a potential misbehavior. We refer to this approach as mempool-based transaction auditing and depict it in Figure 2. Here, Figure 2a shows two observers O_1, O_2 that witnessed three transactions t_{U_1}, t_{U_2} , and t_{U_3} . Both O_1 and O_2 receive t_{U_3} before t_{U_1} and t_{U_2} (denoted as $t_{U_3} \prec t_{U_1}, t_{U_3} \prec t_{U_2}$), while the adversary A proposes a block $\mathcal{B}$ that executes both t_{U_1} and t_{U_2} but omits t_{U_3} .

Unfortunately, while such auditing schemes can emerge as a deterrent against front-running attacks, they can only offer probabilistic guarantees. Namely, it is possible that observers do not always witness a similar order to that of a given miner due to delays, network topology, or other aspects that impact network connectivity. We illustrate this by means of an example in Figure 2b. Here, honest observers O_1 and O_2 receive transactions t_{U_1}, t_{U_2} , and t_{U_3} in different orders due to network jitter.

Research Questions: In this paper, we set forth to explore the precise guarantees that mempool-based transaction auditing approaches can give in blockchain deployments. More specifically, we aim to answer the following fundamental research questions:

- RQ1** What is the probability that up to n independent observers observe the same set of transactions in Bitcoin and Ethereum?
- RQ2** What specific conditions must be satisfied for mempool auditing schemes to conclude with high confidence that no transaction displacement *within blocks* has occurred?
- RQ3** What specific conditions must be met for mempool auditing schemes to reliably determine that no transaction displacement *across different blocks* has taken place? Similarly, when can mempool auditing schemes conclude with high certainty that no censoring has occurred?

RQ4 Based on our analysis, how do threshold fair-ordering schemes that rely on consistent transaction orderings among validators—such as Themis [24]—perform compared to traditional fair-ordering approaches like Pompe [36] and Wendy [26]?

3 Mempool-based Network Auditing

3.1 Network Model

In what follows, we determine the probability that a share of m out of the total n observers agree on two transactions’ orders. First, we determine the probability that a single observer receives two transactions sent ω apart in a certain order (cf. Equation (1)). Then, we generalize this probability to account for multiple observers.

We start by computing the probability that an observer o receives a transaction t_1 before another transaction t_2 that was sent ω time after t_1 . For this, we introduce $\sigma(t)$ denoting the time when a transaction t is sent and $\tau_o(t)$ as the time it takes to deliver transaction t from its sender to observer o . Here, $\tau_o(t)$ is a random variable that follows the distribution⁴ of propagation times $\tau_o(t) \sim \mathcal{D}^{(r)}$. Then, given t_1, t_2 sent at time $\sigma(t_1), \sigma(t_2)$ with $\sigma(t_2) = \sigma(t_1) + \omega$ and received after $\tau_o(t_1), \tau_o(t_2)$ time by observer o , the probability that o receives t_1 before t_2 , i.e., $Pr[t_1 \prec_o t_2]$, is equal to:

$$Pr[t_1 \prec_o t_2] = \int_0^\infty F_{\tau_o(t_1)}(\omega + t) f_{\tau_o(t_2)}(t) dt. \quad (1)$$

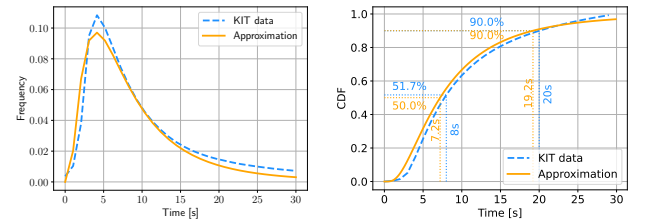
Here, $f_{\tau_o(t)}(T)$ denotes the probability density function (PDF) of the distribution of propagation times $\mathcal{D}^{(r)}$. $F_{\tau_o(t)}(T)$ denotes the cumulative distribution function (CDF) of $\mathcal{D}^{(r)}$, i.e., the probability that $\tau_o(t)$ is less than (or equal to) a value T . Further, it holds $Pr[t_1 \succ_o t_2] = 1 - Pr[t_1 \prec_o t_2]$.

The probability $Pr[t_1 \prec_o t_2]$ is parameterized by ω , i.e., the time difference between sending the two transactions t_1 and t_2 . Evaluating it with a small value ω compares the probabilities of two (almost) equiprobable events. For $\omega = 0$, $Pr[t_1 \prec_o t_2]$ yields a probability of 50%. Moreover, increasing ω causes the integral shown in Equation (1) to monotonically increase.

Next, we leverage Equation (1) to prove Theorem 1, which generalizes the probability to account for multiple observers.

That is, we determine the probability that m out of n observers ($m \leq n$) witness the same order among two transactions. Ultimately, this probability can be leveraged to assess the applicability of an observer-based auditing scheme to detect re-ordering attacks. Here, we assume that the propagation times $\tau_{o_i}(t)$ and $\tau_{o_j}(t)$, i.e., the

⁴We assume that $\mathcal{D}^{(r)}$ is agnostic to a specific network topology or configuration.



(a) PDF of the transactions’ transmis- (b) CDF of the transactions’ transmis-
sion times. sion times.

Figure 3: The PDF/CDF of transaction transmission times based on the KIT dataset (dashed) and its log-normal approximation (solid).

times it takes for a single transaction t to reach two different observers o_i and o_j , are independent random variables for any pair of distinct observers $o_i \neq o_j$. This assumption is based on (1) the observers being randomly positioned within the network, (2) senders are likewise expected to be randomly distributed, and (3) observers are not directly connected, preventing direct snapshot correlations. We further confirm this assumption by empirical measurements (cf. Section 4.1). We now introduce our first main theorem.

THEOREM 1. *Given are two transactions t_1, t_2 with t_2 sent ω time after t_1 and n observers logging network events. Then, the probability that exactly m out of n observers ($m \leq n$) receive transaction t_1 before t_2 is given by:*

$$\begin{aligned} \mathbb{Q}_n(m) &= \Pr \left[m = \left| \left\{ o \in \mathcal{O} \mid (t_1 \prec_o t_2) \right\} \right| \right] \\ &= \binom{n}{m} \left(\int_0^\infty F_{\tau_o(t_1)}(\omega + t) f_{\tau_o(t_2)}(t) dt \right)^m \\ &\quad \cdot \left(1 - \int_0^\infty F_{\tau_o(t_1)}(\omega + t) f_{\tau_o(t_2)}(t) dt \right)^{n-m} \end{aligned} \quad (2)$$

Here, the term $\mathcal{E}_{m/n}^{t_1 \prec t_2}$ denotes the event that exactly m out of n observers receive transaction t_1 before transaction t_2 . It is defined:

$$\mathcal{E}_{m/n}^{t_1 \prec t_2} := m = \left| \left\{ o \in \mathcal{O} \mid (t_1 \prec_o t_2) \right\} \right|. \quad (3)$$

Function $\mathbb{Q}_n(m)$ relates to the probability mass function of the binomial distribution. That is, we model the event $\Pr[t_1 \prec_o t_2]$ for any observer o as a Bernoulli trial. Additionally, we consider any of these Bernoulli trials successful when t_1 is received before t_2 by the respective observer. Then, $\mathbb{Q}_n(m)$ determines the probability that exactly m out of n Bernoulli trials are successful. Here, $\mathbb{Q}_n(m)$ is parameterized by three parameters. These parameters consist of (i) the time difference in sending t_1 and t_2 ω , (ii) the number of observers n , and (iii) the size of the observers' subset that witness the same order of t_1 and t_2 m . Due to space limitations, the proof of Theorem 1 can be found in Section C. Note that $\mathbb{Q}_n(m)$ monotonically increases with increasing ω for fixed values m and n . We evaluate $\mathbb{Q}_n(m)$ in Table 1 and Table 2 based on real-world measurements in the Bitcoin blockchain (cf. Section 3.3).

3.2 Witnessing In-order Events

We now determine the probability that a transaction t_1 has been sent before t_2 conditioned on the event that m out of n observers reported the reception of t_1 before t_2 . This basically allows us to assess the effectiveness of mempool-based auditing schemes.

THEOREM 2. *Given two transactions t_1, t_2 and n observers, the probability that t_1 has been sent before t_2 given that m out of n observers received t_1 before t_2 is:*

$$\begin{aligned} \mathbb{P}_n(m) &= \Pr \left[t_1 \prec t_2 \mid \mathcal{E}_{m/n}^{t_1 \prec t_2} \right] \\ &= \int_0^\infty \left(\frac{\Pr \left[\mathcal{E}_{m/n}^{t_1 \prec t_2} \mid \omega \right] \cdot \Pr[\omega]}{\int_{-\infty}^\infty \Pr \left[\mathcal{E}_{m/n}^{t_1 \prec t_2} \mid \hat{\omega} \right] \cdot \Pr[\hat{\omega}] d\hat{\omega}} \right) d\omega \end{aligned} \quad (4)$$

$\mathbb{P}_n(m)$ estimates the probability that the original sending order of transactions is indeed similar to observers' local event logs. The more observers agree on either the event $t_1 \prec t_2$ or the event $t_2 \prec t_1$, the more likely is ω to be greater or lower than 0. Further, a larger value n , i.e., more observers in the network, allows more confident predictions. That is, the prediction based on a few observers' observations, e.g., $n \leq 3$ is more error-prone than a prediction that is based on many observing auditors, e.g., $n \geq 10$. We evaluate $\mathbb{P}_n(m)$ in Table 3 based on the real-world network behavior measured in Bitcoin (cf. Section 3.3). We observe that $\mathbb{P}_n(m)$ increases in m for a fixed n . The proof for Theorem 2 is given in Section C.

3.3 Instantiating $\mathcal{D}^{(r)}$ in Bitcoin

We now evaluate $\mathbb{Q}_n(m)$ and $\mathbb{P}_n(m)$ in Bitcoin for various combinations of the parameters m, n , and ω . We proceed as follows: first, we use the distribution of transaction propagation times derived from real-world measurements to instantiate the distribution $\mathcal{D}^{(r)}$. By sampling probabilistic propagation times from this distribution, we then estimate $\mathbb{Q}_n(m)$ and $\mathbb{P}_n(m)$ for various numbers of observers n and transactions t_1 and t_2 sent various time differences ω apart.

The KIT Dataset: Neudecker *et al.* [28] provide data from several measurements of the Bitcoin network and its transactions' and blocks' transmission. This data (referred to as the KIT dataset in the sequel) provides an empirical distribution of average transaction reception times within the Bitcoin network. In particular, it evaluates the PDF of the time it takes for a transaction to be forwarded through the network until it is received by an average node.

The distribution of transaction reception times in the KIT dataset is depicted in Figure 3. Here, the probability density function (PDF) of the average transaction's reception times is shown by the blue curve in Figure 3a. We observe that most transactions are received within the first twenty seconds. Specifically, we observe that 90% of all transactions are received after 20 seconds, and 99% of all transactions are received after at most 27.75 seconds. Notice also the long tail of the distribution, indicating that some transactions take over 30 seconds to be received by certain nodes.

Estimating the distribution of transactions' propagation time:

Based on the KIT dataset, we approximate the distribution of transactions' propagation time using the log-normal distribution $\mathcal{LN}(\mu, \sigma^2)$ with mean value $\mu = 1.973$ and variance $\sigma^2 = 0.585$. Our approximation allows us to evaluate the distribution more finely and not rely on data sampled at discrete time points. It also allows us to estimate transaction transmission times beyond the 30 seconds measured in the KIT dataset.

Evaluating $\mathbb{Q}_n$: We then use the log-normal distribution $\mathcal{LN}(1.973, 0.585)$ to evaluate the probability $\mathbb{Q}_n$ for $n = 5$ and $n = 16$ observers. Additionally, we also evaluate the probability $\mathbb{Q}_n$ cumulatively for any subset of at least m out of n observers, i.e., $\sum_{m' \geq m} \mathbb{Q}_n(m')$.

As shown in Table 1, when the time difference ω between transactions is small (i.e., $\omega \leq 5$ seconds), the likelihood that all $m = 5$ observers receive t_1 and t_2 in the correct order is at most 23%. In contrast, when the transactions are spaced by at least $\omega \geq 60$ seconds, this probability exceeds 99%, indicating a strong temporal separation is necessary for consistent ordering across all observers.

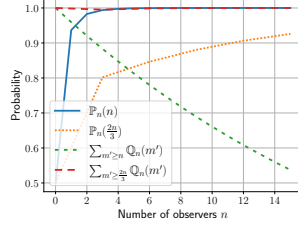
ω	$Q_5(m)$			$\sum_{m' \geq 5} Q_5(m')$	
	$m=3$	$m=4$	$m=5$	$m' \geq 3$	$m' \geq 4$
0	31.2%	15.6%	3.1%	50.0%	18.7%
1	34.0%	21.5%	5.4%	60.9%	26.9%
5	26.8%	39.3%	23.1%	89.2%	62.4%
10	11.1%	37.2%	49.9%	98.2%	87.1%
30	0.2%	7.2%	92.5%	100.0%	99.8%
60	0.0%	0.8%	99.2%	100.0%	100.0%
600	0.0%	0.0%	100.0%	100.0%	100.0%

Table 1: Estimate of $Q_5(m)$ based on the KIT distribution's approximation in Bitcoin.

ω	$Q_{16}(m)$			$\sum_{m' \geq 16} Q_{16}(m')$	
	$m=8$	$m=12$	$m=16$	$m' \geq 8$	$m' \geq 12$
0	19.6%	2.8%	0.0%	59.8%	3.8%
1	17.6%	6.4%	0.0%	76.6%	9.7%
5	2.1%	22.5%	0.9%	99.2%	61.5%
10	0.0%	9.7%	10.8%	100.0%	95.3%
30	0.0%	0.0%	78.0%	100.0%	100.0%
60	0.0%	0.0%	97.4%	100.0%	100.0%
600	0.0%	0.0%	100.0%	100.0%	100.0%

Table 2: Estimate of $Q_{16}(m)$ based on the KIT distribution in Bitcoin.

$n=5$		$n=12$		$n=16$	
m	$P_5(m)$	m	$P_{12}(m)$	m	$P_{16}(m)$
0	0.10%	6	50.98%	9	71.60%
1	4.59%	7	74.79%	10	86.82%
2	29.73%	8	90.63%	11	95.27%
3	72.04%	9	97.66%	12	98.75%
4	96.16%	10	99.66%	14	99.98%
5	99.90%	12	100.0%	16	100.00%

Table 3: valuating $P_n(m)$ using the KIT distribution in Bitcoin.Figure 4: $\sum_{m' \geq m} Q_n(m') / P_n(m)$ w.r.t. the number of observers n .

This intuitively confirms the folklore belief that longer time differences between the sending times of two transactions yield more robustness against accidental re-orderings. Table 2 shows function Q_n evaluated for $n = 16$ observers. Here, we observe that two transactions sent less than $\omega \leq 30$ seconds apart are received by all 16 observers with probability less than 78%. In other words, 22% of the transactions with $\omega \leq 30$ are received by our observers in an inconsistent order. Nevertheless, we observe that receiving two transactions with $\omega \geq 30$ by four out of five (cf. Table 1) and by 12 out of 16 observers (cf. Table 2), i.e., a vast majority of nodes, in order is (nearly) guaranteed.

Overall, we observe that a larger number of observers n reduces the probability that they all witness one certain order, i.e.,

$$n_1 > n_2 \Rightarrow Q_{n_1}(n_1) \leq Q_{n_2}(n_2). \quad (5)$$

However, a larger number of observers witnessing the same order (i.e., m) also increases the precision (or reliability) when detecting order inconsistencies (cf. Table 3).

Naturally, increasing the number of observers n yields more robust results. That is, having more observers decreases the probability that a displacement attack remains unnoticed. However, a high number of observers also reduces the probability that all nodes reach agreement on their mempools' order. As we model the reception of a transaction as a probabilistic process, i.e., sampling the propagation times from a probability distribution $\tau_o(t) \sim \mathcal{D}^{(r)}$, the probability that all observers unanimously report a distinct order reduces as n increases.

Finally, we evaluate both $\sum_{m' \geq m} Q_n(m')$ and $P_n(m)$, which can be used to measure the recall and the precision of mempool auditing schemes, as a function of the number of observers n in the network in Figure 4. On the one hand, we see that increasing the number of observers drastically decreases the probability that all observers witness a pair of two transactions t_1, t_2 in the same order (i.e., $\sum_{m' \geq m} Q_n(m')$). On the other hand, increasing n yields a more precise prediction of the original sending order of t_1 and t_2 (i.e., $P_n(m)$). Relying on a $m = \frac{2n}{3}$ majority improves $\sum_{m' \geq m} Q_n(m')$ but seriously deteriorates probability $P_n(m)$ of the scheme. Additionally, Figure 4 shows that the ideal number of observers for

precise and reliable observations is rather small, e.g., two to six. This contrasts with the large number of observers ($n = 24$) used in Mempool.space [3].

3.4 Displacement across Blocks

We now determine the probability that a transaction gets displaced across blocks. Typically, a displacement to a different block than expected is triggered when a single transaction is not only inverted in order with another transaction, but with a sequence of transactions. Here, we refer to a block $\mathcal{B}$ as an ordered sequence of transactions, i.e., $\mathcal{B}^{(i)} = [t_1^{(i)}, t_2^{(i)}, \dots]$. Displacement across blocks denotes the event that a transaction t should be scheduled according to the deterministic algorithm (cf. Section 2) in block $\mathcal{B}^{(i)}$ but is incorporated in another block $\mathcal{B}^{(j)}$ with $i < j$ appearing after $\mathcal{B}^{(i)}$.

Instances of displacement across blocks are typically detectable by comparing a proposed block with its expected view. Specifically, each observer receives new transactions and stores them in its local mempool, i.e., it maintains a list of unprocessed transactions. Based on its local mempool, every observer has an expected view on the next block, i.e., the expected set of transactions and their order of execution. Then, once a new block $\mathcal{B}^{(i)}$ for slot (i) is mined, we can compare $\mathcal{B}^{(i)}$ with the expected blocks $\mathcal{B}_o^{(i)}$ of the observers.

PROPOSITION 1. *Given n observers that each have an expected view $\mathcal{B}_o^{(i)}$ on a certain block and the actual block $\mathcal{B}^{(i)}$, a transaction t has been displaced across blocks if*

$$\forall o \in \mathcal{O} : t \in \mathcal{B}_o^{(i)} \wedge t \in \mathcal{B}^{(j)}, j > i. \quad (6)$$

In other words, if a transaction t that is apparent in the expected blocks $\mathcal{B}_o$ of all observers but is missing in $\mathcal{B}$, then t appears to have been displaced across blocks. However, as shown in Section 3.1, two honest nodes might receive transactions in a different order. That is, not every transaction that is expected in $\mathcal{B}_o$ and missing in $\mathcal{B}$ is an intentional displacement. In fact, the probability that a transaction t is omitted by an honest miner without any intention to displace t can be determined as shown in Theorem 3.

THEOREM 3. *Given a transaction t and n observers, the probability that a sequence of transactions $(t_i)_i$, which have all been sent at least ω time after t , i.e., $\sigma(t_i) \geq \sigma(t) + \omega$, $\forall i$, are all received before t is:*

$$\Pr \left[\bigwedge_{o \in \mathcal{O}} \left(\bigwedge_i t_i \prec_o t \right) \right] = \left(1 - F_{\tau_o(t)}(\tau_C) \right)^n \quad (7)$$

Recently received transactions are excluded from this definition; a transaction must be received at least τ_C time ago before it is considered to be displaced (see full proof in Section C).

4 Empirical Validation

In this section, we empirically evaluate our analysis presented in Section 3 using two popular blockchains, Bitcoin and Ethereum. Specifically, we evaluate the probability that at least m out of n blockchain nodes receive t_1 before t_2 , i.e., $\sum_{m' \geq m} \mathbb{Q}_n(m')$.

For this purpose, we set up 16 Bitcoin and Ethereum nodes in different regions around the globe (cf. Table 6 in Appendix B) to record transactions' reception times and their order of arrival. Our nodes were hosted by Hetzner⁵ cloud [21]; each server featured four dedicated vCPU kernels, 16GB RAM, SSD storage, an AMD EPYC-Milan processor, and was reachable by an IPv4 address. All measurements were conducted on servers running Ubuntu 24.04 LTS. We used *bitcoind 25.0* as our Bitcoin client and the *geth 1.15.11* client for Ethereum. Throughout our measurements, each observer logged all incoming transactions and their exact reception time between 30.06. and 08.07.2025, by a total of 16 geographically dispersed observers (see Table 7 in Appendix B).

4.1 Real-world Evaluation in Bitcoin

In Bitcoin, our $n = 16$ observers recorded in total 3 134 824 transactions over the course of 196 hours. Before evaluating our findings, we first report the detailed reception statistics in Table 4. While one would expect all transactions to be eventually received by any node in the network, we observe that some transactions were only partially received by a subset of our 16 observers. Here, we observe that out of all transactions, only 93.93% were received by every observer. Further, 99.08% of all transactions were received by a subset of at least $m \geq 12$ nodes, and 99.34% were received by a majority of at least $m \geq 9$ nodes. On average, each node received $98.9\% \pm 0.15\%$ of the recorded transactions. We believe that the fraction of transactions received only by a subset of our 16 observers is due to transactions that have already circulated through the network for a while and are not forwarded again by all nodes, or due to transactions that have not reached all 16 nodes yet. Based on the recorded transactions, we compute the pairwise Mutual Information (MI) score between observers' reception times, and find an average MI of 0.15 nats, indicating that reception times are practically independent, confirming our assumption in Section 3.1.

We now use our recorded data to empirically validate our analysis. First, we determine an empirical estimate of $\mathbb{Q}_n(m)$. That is, we are interested in the fraction of transaction pairs for which our observers agree on their order of reception. For that purpose, we sample 100 000 random pairs of transactions $(t_0^{(i)}, t_1^{(i)})$ with $i \in I := [1, 100\,000]$, i.e., 200 000 transactions that are sampled uniform at random from our recorded transaction sets and randomly paired. For each pair i , we then determine the number of observers that agree on an order, i.e.,

$$C_{order}(m) = \frac{1}{|I|} \cdot \left| \left\{ i \in I \mid m \geq X(i) \right\} \right| \quad (8)$$

with $X(i)$ denoting the number of observers that witnessed the most frequent order of reception:

$$X(i) = \max_{j \in \{0,1\}} \left| \left\{ o \in \mathcal{O} \mid t_j^{(i)} \prec_o t_{1-j}^{(i)} \right\} \right|. \quad (9)$$

⁵We initially used AWS but faced stability issues with nodes crashing, so we used Hetzner for more reliable measurements.

In other words, $C_{order}(m)$ is the fraction of transaction pairs that have been received in the same order by at least m out of n observers. Hence, $C_{order}(m)$ is the empirical equivalent to our probability $\sum_{m' \geq m} \mathbb{Q}_n(m')$ ⁶.

Table 4 shows the evaluation results of $C_{order}(m)$ with respect to m in Bitcoin. Here, we sampled the 100,000 transaction pairs from (1) the complete dataset (*All Tx*s) and (2) the subset of transactions that were fully received by all 16 observers (*FR Tx*s). Our results show that 88.23% of the randomly drawn transaction pairs from *All Tx*s are witnessed in the same order by all 16 observers. By restricting our dataset to transactions from *FR Tx*s, we observe that 99.98% of the sampled transaction pairs were received in order.

We then compare our empirical findings in the Bitcoin blockchain with the probabilities from Table 2 (cf. Section 3.3) in Figure 5. Here, the dashed lines denote the cumulative probability that at least m out of n observers receive two transactions sent ω seconds apart in the rightful order, i.e., $\sum_{m' \geq m} \mathbb{Q}_n(m')$ (cf. Section 3.1). Further, the solid, red, and lilac lines denote our measurement results as shown in Table 4, i.e., $C_{order}(m)$. The dashed lines show that the probability that $m \geq 14$ observers witness the same order is approximately 97.5% when $\omega = 20$ while our empirical results reveal that transaction pairs in practice are received in the same order by $m \geq 14$ observers with a comparable probability of 96.63% shown by the red plot (*All Tx*s). Further, the violet plot (*FR Tx*s) reveals the probability of fully-received transactions to be almost guaranteed. This conforms with our analytical results for $\omega \geq 60$.

Note that our empirical results on the Bitcoin network are based on measurements conducted over the course of 196 hours. Hence, on average, our randomly sampled transaction pairs used to determine $C_{order}(m)$ have a time difference of $\omega \approx 236\,028$ seconds.

In Appendix D, we additionally present selected results using observers located exclusively in the US and Europe, reflecting the current deployment landscape in which the majority of Bitcoin and Ethereum observers⁷—as well as most Mempool.space [3] observers—are situated in these regions. Our findings remain consistent regardless of whether observers are restricted to the US/EU or distributed more globally, including regions such as Asia. Furthermore, our results are robust across different measurement periods.⁸

4.2 Real-world Evaluation in Ethereum

Our evaluation results in the Ethereum blockchain are shown in Table 4 and encompass a period of 196 hours, during which we recorded a total of 6 552 228 Ethereum transactions. Out of all analyzed transactions, nearly 87.5% were received by all 16 nodes, 95.26% were received by a subset of 12 nodes, and 96.29% by a majority of at least nine nodes. Each node received, on average, $96.32\% \pm 0.83\%$ of the recorded Ethereum transactions. However, one of the 16 observers received only 89.67% of the transactions.

Based on this analysis, we measure $C_{order}(m)$ (cf. Equation (8)), i.e., the empirical estimate of $\sum_{m' \geq m} \mathbb{Q}_n(m')$, in Ethereum. Our

⁶Since the ground truth for the sending time τ of transactions is unavailable, we cannot determine the exact sending time of individual transactions. However, because sending and reception times follow similar distributions, discrepancies in propagation times are expected to average out across 100,000 pairs.

⁷According to data from bitnodes.io, ethernodes.org, and mempool.space.

⁸In Appendix E, we include data from an initial measurement campaign conducted in October 2024 using 20 Bitcoin nodes, which are also consistent with our findings.

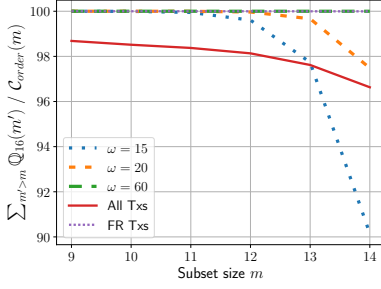


Figure 5: Comparison of $\sum_{m' \geq m} Q_{16}(m')$ (dashed lines) with the empirically measured $C_{order}(m)$ (solid/dotted lines) using the complete dataset (*All TxS*) and the set of transactions received by all observers (*FR TxS*) measured by 16 Bitcoin observers.

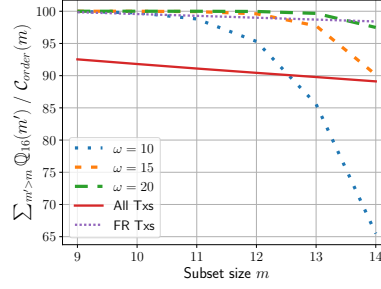


Figure 6: Comparing $\sum_{m' \geq m} Q_{20}(m')$ (dashed lines) with the empirical measurement $C_{order}(m)$ (solid/dotted lines) using the full dataset (*All TxS*) and the transactions received by all observers (*FR TxS*) measured by 16 Ethereum observers.

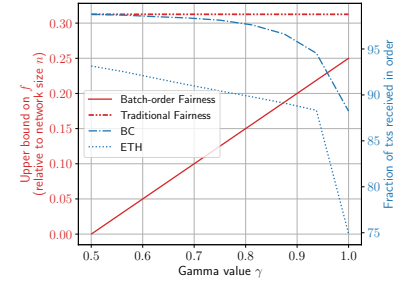


Figure 7: Comparison of batch-order fairness schemes (e.g., Themis[24]) vs. traditional fair-ordering schemes (e.g., Pompe [36] and Wendy [26]). We see that Themis only offers strong guarantees to a fraction of the transactions.

m	Bitcoin (3 134 824 TxS)			Ethereum (6 552 228 TxS)		
	Received TxS	$C_{order}(m)$ All TxS	$C_{order}(m)$ FR TxS	Received TxS	$C_{order}(m)$ All TxS	$C_{order}(m)$ FR TxS
≥ 8 nodes	99.38%	98.77%	100.0%	96.49%	93.21%	100.00%
≥ 9 nodes	99.34%	98.68%	100.0%	96.29%	92.57%	99.86%
≥ 10 nodes	99.26%	98.52%	100.0%	96.08%	91.86%	99.57%
≥ 11 nodes	99.19%	98.37%	100.0%	95.87%	91.15%	99.29%
≥ 12 nodes	99.08%	98.13%	100.0%	95.26%	90.47%	99.00%
≥ 13 nodes	98.83%	97.62%	100.0%	95.63%	89.82%	98.71%
≥ 14 nodes	98.34%	96.63%	100.0%	95.34%	89.14%	98.42%
≥ 15 nodes	97.28%	94.53%	99.99%	94.96%	88.33%	98.13%
≥ 16 nodes	93.93%	88.23%	99.98%	87.49%	74.92%	97.84%

Table 4: Received TxS denotes the percentage of transactions recorded by at least m out of $n = 16$ observers in Bitcoin and Ethereum. $C_{order}(m)$ denotes the fraction of transactions received in a consistent order by m nodes. Results labeled with *FR TxS* are based on the intersection of transactions received by all 16 observers. Results labeled with *All TxS* include all transactions.

results are shown in Table 4. Here, we observe that the fraction of transaction pairs sampled from *All TxS* that were witnessed in the same order by all observers is only 74.92%, and the fraction witnessed by a subset of at least 12 out of 16 observers is only 90.47%. Recall that our Ethereum nodes did not receive all recorded transactions; needless to mention, nodes cannot take a stance on the pairwise ordering of two transactions if one or both were not received. If we sample transaction pairs from *FR TxS*, we observe that 97.84% of these transactions were received in the same order by all 16 observers, and roughly $\geq 99.0\%$ by at least 12 observers.

Note that there exists no ground truth for the distribution of our transactions' propagation times, which prevents us from directly comparing our empirical results on $C_{order}(m)$ with its analytical counterpart. However, Figure 6 shows a best-effort approximation on the comparison between $C_{order}(m)$ (solid plots) and $\sum_{m' \geq m} Q_n(m')$ (dashed plots) instantiated with the distribution of propagation times $\mathcal{D}^{(r)} = \mathcal{LN}(1.973, 0.585)$ (cf. Section 3.3). Our empirical results for $C_{order}(m)$ using *All TxS* resemble those for $\sum_{m' \geq m} Q_n(m')$ with $\omega \in [10, 15]$ seconds. Similarly, we observe that our results on $C_{order}(m)$ using only transactions from *FR TxS* closely follow our empirical results recorded in Bitcoin and conform to $\sum_{m' \geq m} Q_n(m')$ with $\omega \geq 20$.

Specifically, our results show that transactions received by all observers are received in a consistent order among our observers with high probability. This confirms our analytical results for large time differences $\omega \gg 600$ shown in Table 1 and Table 2, respectively.

Overall, our results suggest a more consistent reception of transactions in the Bitcoin network compared to the Ethereum network. That is, 99.34% of the Bitcoin transactions and 96.29% of the Ethereum transactions were received by a majority, i.e., $m \geq 9$, of observers. In general, we suspect the unreliable delivery of transactions is caused by: (1) the connectivity of our observers in the respective blockchain's network, (2) the limited duration of our measurement, or (3) unstable network connections of our cloud nodes. Furthermore, we suspect that the differences between Bitcoin and Ethereum are caused by messages sent over off-chain networks, or over second-layer applications in Ethereum (cf. Appendix 6).

5 Interpretation of Findings

How likely are independent observers to witness the same set of transactions? (RQ1). In Bitcoin, 93.93% of the recorded transactions were observed by all 16 observers, and 99.08% were observed by at least 12 out of 16. In contrast, in Ethereum, only 87.49% of the recorded transactions were received by all observers, and 95.26% by at least 12 out of 16.

Can Mempool Auditing dismiss displacement attacks within blocks? (RQ2). Our results indicate that detecting transaction displacements primarily depends on the sending time difference ω and the number of observers n . Larger ω values increase detection certainty. Using the KIT distribution of Bitcoin propagation times $\mathcal{D}^{(r)}$, Tables 1 and 2 show that for $\omega \geq 30$, most observers (4/5 and 12/16, respectively) agree on transaction order. Thus, if two transactions are sent within 30 seconds, displacement attacks can be reliably detected. Empirical measurements confirm this: while some transactions are inconsistently received, pairs observed by all 16 observers almost always preserve order. In contrast, displacements with short gaps ($\omega \leq 30$ s) cannot always be detected.

Can Mempool Auditing help dismiss displacement across blocks? (RQ3). Our third research question examines when transaction displacement across blocks can be confidently detected. Assuming all honest, rational nodes (i) order mempool transactions by arrival time and (ii) apply a public deterministic scheduling algorithm, one can estimate execution order by monitoring arrivals and comparing the block's contents to a node's expected set.

However, our results (Table 4) show limited certainty: 0.92% of Bitcoin transactions and 4.74% of Ethereum transactions were missed by 4/16 nodes. Thus, cross-block displacement cannot generally be detected with certainty (RQ3).

Censorship is implicitly covered in this analysis, since omitting a transaction when minting a block is equivalent to displacing it across blocks. Note that protocol parameters like block rate do not change this conclusion but affect interpretation: cross-block displacement is harder to detect in high block-rate chains (e.g., Ethereum) than in low block-rate ones (e.g., Bitcoin) due to the shorter block interval.

How do batch-order fair-ordering schemes perform compared to traditional fair-ordering approaches? (RQ4) Kelkar *et al.* [25] introduced *batch-order fairness*, which ensures fair ordering if a fraction γ of honest nodes ($\frac{1}{2} < \gamma \leq 1$) receive two transactions in the same order. This guarantee is stronger than traditional fair-ordering properties (e.g., Pompe [36], Wendy [26]), which allow a transaction t_2 to be output even if t_1 , observed earlier by all, is withheld—thus enabling censorship. Themis [24], a prominent fair-ordering scheme, achieves γ -batch-order fairness but with reduced fault tolerance: it tolerates $f < n \frac{2\gamma-1}{2(\gamma+1)}$ corrupted nodes, compared to the traditional $f < \frac{n}{3}$. Our analysis provides the first empirical assessment of γ , relating the adversarial bound $f(\gamma)$ to the observed fraction of nodes that agree on order ($\gamma = m/n$). Figure 7 shows that the strongest guarantee ($\gamma = 1$, tolerating $f < n/4$) applies to 88.23% of Bitcoin and 74.92% of Ethereum transactions. A weaker guarantee ($\gamma = 0.75$, 12/16 observers agreeing) covers 98.13% of Bitcoin and 90.47% of Ethereum transactions, but tolerates only $f < 0.143n$. Overall, Figure 7 demonstrates that while batch-order fair-ordering schemes can provide interesting fairness guarantees, their applicability is limited to a subset of transactions.

6 Threats to Validity

Non-uniform Distribution of Observers: Our analysis assumes that observers are randomly distributed and not directly connected, ensuring that their information reception times are independent (Section 3.1). If this assumption does not hold, the reception times recorded by different observers for the same transactions may become correlated, which would further reduce the reliability of the auditing process, as the observed views would be more biased.

Off-chain Network Events and Flash Loans. A limitation of mempool auditing is the inability to detect transactions that are exchanged off-chain, such as flash loan transactions. First, since observers of the blockchain network do not receive off-chain transactions, they cannot log their reception times. Second, as off-chain transactions are (exclusively) queued up by a dedicated flash loan server and maintained in the server’s mempool, assembling a block with either on-chain or off-chain transactions will naturally be recognized as an instance of transactions’ displacements by other observers. While it is challenging to obtain reliable estimates of transaction counts and/or propagation characteristics from existing second-layer (L2) solutions, we measured the fraction of blobs, i.e., transaction types introduced in EIP-4844 [19, 35] and predominantly used to aggregate/pack L2 commitments onto the L1 blockchain, among all transactions in 500 randomly selected Ethereum blocks generated on 25.11.2025. Our measurements show that L2 blobs

account for up to 6% of L1 transactions. This estimate, together with Ethereum’s higher throughput of $2.1\times$ that of Bitcoin, could explain the slightly lower consistency observed in Ethereum (Section 4.2).

Notice, however, that mempool auditing can be easily extended to monitor off-chain transactions as well to address this limitation. This can be achieved, e.g., by accepting connections from users that operate off-chain to our on-chain observers. Such observers would then offer an API to, e.g., users with flash loan transactions. Once a user intends to issue an off-chain transaction, i.e., a transaction that is sent to a flash loan server, the user additionally forwards it to a set of (off-chain) observers.

Network & Block Stuffing Attacks. A third limitation of mempool auditing arises from an adversary’s ability to interfere with the propagation of honest transactions in the network. A network adversary that delays transaction dissemination—e.g., by dropping incoming transactions or by flooding the network with a large volume of transactions—may distort transaction reception times and potentially manipulate the observed ordering for its own benefit.

A concrete example is the block-stuffing attack [29], in which the adversary injects a burst of high-fee transactions that saturate a block’s gas limit, preventing the inclusion of other transactions. By flooding the network with such transactions, the adversary can delay the execution of honest users’ transactions. If an observer O is targeted by such an attack, real-time auditing—where decisions must be made within fixed time windows—may produce delayed or inconsistent mempool snapshots relative to those observed by other observers. This discrepancy can lead to cross-block displacement. Block-stuffing attacks can be mitigated through fee-rate “smoothing” [5, 8, 16]. For example, Child Pays For Parent [8] enables boosting low-fee transactions by attaching a higher-fee child transaction that spends their outputs. Such mechanisms can be incorporated in the deterministic algorithm (cf. Det. Alg., Section 2) to enforce consistent behavior across miners.

Addressing the broader class of network-level attacks typically requires robust communication protocols that withstand adversarial interference [4]. Possible measures include increasing node fanout in graphs to ensure better connectivity among nodes, etc.

7 Conclusion

In this paper, we present the first in-depth analysis of the effectiveness of mempool-auditing in detecting transaction displacement attacks by malicious miners. Our analysis specifically examines the guarantees that mempool-based detection offers in realistic scenarios, such as those in the current Bitcoin and Ethereum ecosystems. Our findings indicate that while mempool auditing can assist in identifying transaction displacement attacks by miners, relying solely on this method to assess miners’ reputations—as some in the community advocate—does not provide definitive proof of malicious actions. In fact, our results suggest that mempool auditing may lead to false accusations against miners, with a probability exceeding 40% in some settings.

Acknowledgments

This work is funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy - EXC 2092 CASA - 390781972.

References

- [1] Blockchain.com. Accessed 2025. Blockchain.com Bitcoin Explorer. <https://www.blockchain.com/explorer/mempool/btc>.
- [2] Etherscan. Accessed 2025. Etherscan - Ethereum Blockchain Explorer. <https://etherscan.io/txsPending>.
- [3] The Mempool Open Source Project. Accessed 2025. The Mempool Open Source Project. <https://mempool.space/>.
- [4] Jannik Albrecht, Sébastien Andreina, Frederik Armknecht, Ghassan Karame, Gorgia Azzurra Marson, and Julian Willingmann. 2024. Larger-scale Nakamoto-style Blockchains Don't Necessarily Offer Better Security. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 2161–2179. doi:10.1109/SP54263.2024.00227
- [5] Soumya Basu, David A. Easley, Maureen O'Hara, and Emin Gün Sirer. 2019. Towards a Functional Fee Market for Cryptocurrencies. *CoRR* abs/1901.06830 (2019). arXiv:1901.06830 <http://arxiv.org/abs/1901.06830>
- [6] Carsten Baum, James Hsin-yu Chiang, Bernardo David, Tore Kasper Frederiksen, and Lorenzo Gentile. 2022. SoK: Mitigation of Front-Running in Decentralized Finance. In *Financial Cryptography and Data Security. FC 2022 International Workshops - CoDecFin, DeFi, Voting, WTSC, Grenada, May 6, 2022, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 13412)*, Shin'ichiro Matsuo, Lewis Gudgeon, Arian Klages-Mundt, Daniel Perez Hernandez, Sam Werner, Thomas Haines, Aleksander Essex, Andrea Bracciali, and Massimiliano Sala (Eds.). Springer, 250–271. doi:10.1007/978-3-031-32415-4_17
- [7] Bitcoin. 2025. *Bitcoin Core*. <https://github.com/bitcoin/bitcoin>
- [8] Bitcoin Optech. 2026. *Child Pays For Parent (CPFP)*. <https://bitcoinops.org/en/topics/cpfp/>
- [9] Blockchain.com. Accessed 2025. Blockchain.com Ethereum Explorer. <https://www.blockchain.com/explorer/mempool/eth>.
- [10] Blockchain.com. Accessed 2025. Hashrate Distribution in Bitcoin. <https://www.blockchain.com/explorer/charts/pools>.
- [11] Blockstream Explorer. Accessed 2025. Blockstream Explorer. <https://blockstream.info/tx/recent>.
- [12] Lorenz Breidenbach, Philip Daian, Florian Tramèr, and Ari Juels. 2018. Enter the Hydra: Towards Principled Bug Bounties and Exploit-Resistant Smart Contracts. In *27th USENIX Security Symposium, USENIX Security 2018, Baltimore, MD, USA, August 15-17, 2018*, William Enck and Adrienne Porter Felt (Eds.). USENIX Association, 1335–1352. <https://www.usenix.org/conference/usenixsecurity18/presentation/breidenbach>
- [13] Lorenz Breidenbach, Tyler Kell, Stephane Gosselin, and Shayan Eskandari. 2018. LibSubmarine. <https://libsubmarine.org>.
- [14] BTC Explorer by Redot. Accessed 2025. Btcsan. <https://btcsan.org/tx/recent>.
- [15] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. 2019. Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges. *CoRR* abs/1904.05234 (2019). arXiv:1904.05234 <http://arxiv.org/abs/1904.05234>
- [16] Sankarshan Damle, Varul Srivastava, and Sujit Gujar. 2024. No Transaction Fees? No Problem! Achieving Fairness in Transaction Fee Mechanism Design. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems, AAMAS 2024, Auckland, New Zealand, May 6-10, 2024*, Mehdi Dastani, Jaime Simão Sichman, Natasha Alechina, and Virginia Dignum (Eds.). International Foundation for Autonomous Agents and Multiagent Systems / ACM, 2228–2230. doi:10.5555/3635637.3663116
- [17] Shayan Eskandari, Seyedehmahsa Moosavi, and Jeremy Clark. 2019. SoK: Transparent Dishonesty: Front-Running Attacks on Blockchain. In *Financial Cryptography and Data Security - FC 2019 International Workshops, VOTING and WTSC, St. Kitts, St. Kitts and Nevis, February 18-22, 2019, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 11599)*, Andrea Bracciali, Jeremy Clark, Federico Pintore, Peter B. Rønne, and Massimiliano Sala (Eds.). Springer, 170–189. doi:10.1007/978-3-030-43725-1_13
- [18] Ethereum. 2025. *Go Ethereum*. <https://github.com/ethereum/go-ethereum#>
- [19] Ethereum EIPs - GitHub Repository. 2024. *EIP 4844*. <https://github.com/ethereum/EIPs/blob/master/EIPS/eip-4844.md>
- [20] Etherscan. Accessed 2025. Block Producers in Ethereum. <https://etherscan.io/dashboards/block-producers>.
- [21] Hetzner Online. Accessed 2025. Hetzner Cloud. <https://www.hetzner.com/cloud/>.
- [22] Jannik Albrecht, Ghassan Karame. January 2026. Artifact for the paper "On the Effectiveness of Mempool-based Transaction Auditing". <https://github.com/RUB-InfSec/mempool-based-transaction-auditing>.
- [23] Mahimna Kelkar, Soubhik Deb, and Sreeram Kannan. 2022. Order-Fair Consensus in the Permissionless Setting. In *APKC '22: Proceedings of the 9th ACM on Asia Public-Key Cryptography Workshop, APKC@AsiaCCS 2022, Nagasaki, Japan, 30 May 2022*, Jason Paul Cruz and Naoto Yanai (Eds.). ACM, 3–14. doi:10.1145/3494105.3526239
- [24] Mahimna Kelkar, Soubhik Deb, Sishan Long, Ari Juels, and Sreeram Kannan. 2023. Themis: Fast, Strong Order-Fairness in Byzantine Consensus. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM, 475–489. doi:10.1145/3576915.3616658
- [25] Mahimna Kelkar, Fan Zhang, Steven Goldfeder, and Ari Juels. 2020. Order-Fairness for Byzantine Consensus. In *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 12172)*, Daniele Micciancio and Thomas Ristenpart (Eds.). Springer, 451–480. doi:10.1007/978-3-030-56877-1_16
- [26] Klaus Kursawe. 2020. Wendy, the Good Little Fairness Widget: Achieving Order Fairness for Blockchains. In *AFT '20: 2nd ACM Conference on Advances in Financial Technologies, New York, NY, USA, October 21-23, 2020*. ACM, 25–36. doi:10.1145/3419614.3423263
- [27] Patrick McCorry, Alexander Hicks, and Sarah Meiklejohn. 2018. Smart Contracts for Bribing Miners. In *Financial Cryptography and Data Security - FC 2018 International Workshops, BITCOIN, VOTING, and WTSC, Nieuwpoort, Curaçao, March 2, 2018, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 10958)*, Aviv Zohar, Ittay Eyal, Vanessa Teague, Jeremy Clark, Andrea Bracciali, Federico Pintore, and Massimiliano Sala (Eds.). Springer, 3–18. doi:10.1007/978-3-662-58820-8_1
- [28] Till Neudecker, Philipp Andelfinger, and Hannes Hartenstein. 2016. Timing Analysis for Inferring the Topology of the Bitcoin Peer-to-Peer Network. In *Proceedings of the 13th IEEE International Conference on Advanced and Trusted Computing (ATC)*.
- [29] Onur Solmaz. Accessed 2025. The Anatomy of a Block Stuffing Attack. <https://solmaz.io/2018/10/18/anatomy-block-stuffing/>.
- [30] Kaihua Qin, Liyi Zhou, and Arthur Gervais. 2022. Quantifying Blockchain Extractable Value: How dark is the forest?. In *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022*. IEEE, 198–214. doi:10.1109/SP46214.2022.9833734
- [31] Kaihua Qin, Liyi Zhou, Benjamin Livshits, and Arthur Gervais. 2021. Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit. In *Financial Cryptography and Data Security - 25th International Conference, FC 2021, Virtual Event, March 1-5, 2021, Revised Selected Papers, Part I (Lecture Notes in Computer Science, Vol. 12674)*, Nikita Borisov and Claudia Diaz (Eds.). Springer, 3–32. doi:10.1007/978-3-662-64322-8_1
- [32] The Mempool Open Source Project. Accessed 2025. Litecoin Space Explorer. <https://litecoinspace.org/>.
- [33] Tokenview Ethereum Explorer. Accessed 2025. Tokenview. <https://eth.tokenview.io/en/pending>.
- [34] Christof Ferreira Torres, Ramiro Camino, and Radu State. 2021. Frontrunner Jones and the Raiders of the Dark Forest: An Empirical Study of Frontrunning on the Ethereum Blockchain. In *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, Michael D. Bailey and Rachel Greenstadt (Eds.). USENIX Association, 1343–1359. <https://www.usenix.org/conference/usenixsecurity21/presentation/torres>
- [35] Vitalik Buterin, Dankrad Feist, Diederik Loerakker, George Kadianakis, Matt Garnett, Mofi Taiwo, Ansgar Dietrichs. 2026. *EIP-4844: Shard Blob Transactions*. <https://eips.ethereum.org/EIPS/eip-4844>
- [36] Yunhao Zhang, Srinath T. V. Setty, Qi Chen, Lidong Zhou, and Lorenzo Alvisi. 2020. Byzantine Ordered Consensus without Byzantine Oligarchy. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 633–649. <https://www.usenix.org/conference/osdi20/presentation/zhang-yunhao>
- [37] Liyi Zhou, Kaihua Qin, and Arthur Gervais. 2021. A2MM: Mitigating Frontrunning, Transaction Reordering and Consensus Instability in Decentralized Exchanges. *CoRR* abs/2106.07371 (2021). arXiv:2106.07371 <https://arxiv.org/abs/2106.07371>
- [38] Liyi Zhou, Xihan Xiong, Jens Ernstberger, Stefanos Chaliasos, Zhipeng Wang, Ye Wang, Kaihua Qin, Roger Wattenhofer, Dawn Song, and Arthur Gervais. 2023. SoK: Decentralized Finance (DeFi) Attacks. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 2444–2461. doi:10.1109/SP46215.2023.10179435

A Additional Related Work

The literature features a number of contributions that analyze transaction displacement attacks and propose various mitigation approaches. An overview of common displacement attacks and defenses is given by Eskandari *et al.* [17], Baum *et al.* [6], and Zhou *et al.* [38]. In particular, Eskandari *et al.* [17] categorize existing attack vectors into three classes *displacement*, *insertion*, and *suppression*, all of which fall within our broader definition of displacement attacks. Existing solutions to mitigate the threat typically either (i) prevent

	Detection Tool	LF	ETA	Additional Notes
Bitcoin	MemPool Open Source Project [3]	✓	✓ ¹	$n = 24$; ¹ “Block audits” comparing the expected block with the actual block (cf. Figure 8)
	Blockstream Explorer [11]	✓	✓	Simple log of transactions with ETA
	Btscan [14]	✓	✓	Simple log of transactions with ETA
	Blockchain.com [1]	✓	✗	Precise reception times
Ethereum	Tokenview [33]	✓	✗	Precise reception times
	Etherscan [2]	✓	✓	Precise reception times & estimated transaction confirmation duration
	Blockchain.com [9]	✓	✗	Precise reception times

Table 5: Summary of existing ad-hoc detection tools. Services with the label *LF* provide a live feed of recently received transactions. The label *ETA* denotes that the service provides an expected time for transactions to appear in a block. From the analyzed tools, MemPool.space [3] is the only one that publicly disclosed the number of its observers.

the leakage of sensitive data until the corresponding transaction is executed or (ii) establish an order of execution that is considered fair. The first type of defenses *commit & reveal* is a commitment scheme and belongs to the former class. It attempts to thwart displacement and censoring attacks by blinding the transaction’s data, i.e., its value and/or receiver. Then, after the transaction’s execution is scheduled and cannot be modified, the hidden data is revealed. Libsubmarine [13] is an open-source smart contract library that belongs to the class *commit & reveal* schemes. It leverages a technique called Submarine Commitments proposed and first analyzed by Breidenbach *et al.* [12]. Another line of work by Kursawe [26] and Kelkar *et al.* [23–25] attempts to thwart front-running by establishing a fixed ordering of transactions’ execution. Both Kursawe and Kelkar *et al.* propose to establish a *fair ordering* of transactions. In particular, Kelkar *et al.* [25] proposed a consensus property coined *transaction order-fairness* in the permissioned setting. In subsequent work, Kelkar *et al.* [23] extended this property to the permissionless setting. Finally, Kelkar *et al.* [24] propose Themis—an integrative scheme to accomplish a fair ordering of transactions in permissioned blockchains based on the notion of batch-order fairness [25] and reliant on a consistent ordering of transactions.

Unfortunately, these existing solutions are impractical for existing blockchains. Namely, they either cannot support real-time detection/prevention or are not generically applicable to different blockchains/applications. For instance, *commit & reveal* schemes such as submarine commitments [12, 13] cannot function in real-time. Since the transaction’s data is sealed until the transaction’s execution is scheduled, the processing time per transaction is increased. In particular, critical transactions are negatively affected by increased processing time. In contrast, mitigation schemes that leverage fair ordering and input aggregation approaches can realize real-time functionality. We observe that batch-order fair ordering schemes [23–26] rely on liveness, synchronized clocks among miners, and assume a consistent ordering of transactions as received by the orderers—an assumption that, to date, has not been empirically validated due to the absence of thorough analysis on this subject.

B Observer Locations

Table 6 shows the distribution of the complete set of 16 nodes, and Table 7 shows the locations for the subset of 12 nodes located exclusively in the US and Europe. Note that we report additional results

State/Country	Network zone	#Nodes
Virginia, USA	us-east	2
Oregon, USA	us-west	2
Germany	eu-central	4
Finland	eu-north	4
Singapore	asia	4
		16

Table 6: The locations of all our observers.

State/Country	Network zone	#Nodes
Virginia, USA	us-east	2
Oregon, USA	us-west	2
Germany	eu-central	4
Finland	eu-north	4
		12

Table 7: Our subset of 12 observers that are located in the US and Europe exclusively.

on the reception rates of our observer nodes located exclusively in the US and Europe in Section D.

C Proofs

PROOF OF THEOREM 1. We start our proof by introducing the two sets O_1 and O_2 . We assign all observers to either of these two sets such that $n = |O_1| + |O_2|$. That is, all observers in O_1 receive t_1 before t_2 and all observers in O_2 receive t_2 before t_1 . We first determine the number of combinations for sampling m out of n observers for O_1 . That is, given n observers, there exist exactly $\binom{n}{m}$ distinct combinations of observers to form O_1 with $|O_1| = m$.

Next, we rearrange the above equation using the fact that O_1 and O_2 are disjoint sets:

$$\begin{aligned}
 Pr \left[m = \left| \left\{ o \in O \mid (t_1 \prec_o t_2) \right\} \right| \right] & \\
 = \binom{n}{m} \cdot Pr \left[\bigwedge_{o \in O_1} t_1 \prec_o t_2 \right] \cdot Pr \left[\bigwedge_{o \in O_2} t_1 \succ_o t_2 \right] & \quad (10)
 \end{aligned}$$

Assuming that $\tau_{o_i}(t_x)$ for $\forall o_i \in O, \forall x$ are drawn from the same distribution with $\tau_{o_i}(t_x)$ being independent from $\tau_{o_j}(t_x)$ for $i \neq j$, it holds:

$$Pr \left[\bigwedge_{o \in O} t_1 \prec_o t_2 \right] = Pr [t_1 \prec_o t_2]^{|O|} \quad (11)$$

Finally, applying Equation (1) yields:

$$Pr [t_1 \prec_o t_2]^{|O|} = \left(\int_0^\infty F_{\tau_{o_i}(t_1)}(\omega + \iota) f_{\tau_{o_i}(t_2)}(\iota) d\iota \right)^{|O|} \quad (12)$$

Combining these observations, the probability that any m out of n observers receive transaction t_1 before t_2 is given by:

$$\begin{aligned} \mathbb{Q}_n(m) &= \Pr \left[m = \left| \left\{ o \in O \mid (t_1 \prec_o t_2) \right\} \right| \right] \\ &= \binom{n}{m} \left(\int_0^\infty F_{\tau_o(t_1)}(\omega + t) f_{\tau_o(t_2)}(t) dt \right)^m \\ &\quad \cdot \left(1 - \int_0^\infty F_{\tau_o(t_1)}(\omega + t) f_{\tau_o(t_2)}(t) dt \right)^{n-m} \end{aligned} \quad (13)$$

□

PROOF OF THEOREM 2. We rearrange the above probability as follows:

$$\Pr \left[t_1 \ll t_2 \mid \mathcal{E}_{m/n}^{t_1 \prec t_2} \right] = \int_{-\infty}^0 \Pr \left[\omega \mid \mathcal{E}_{m/n}^{t_1 \prec t_2} \right] d\omega \quad (14)$$

The probability distribution of ω conditioned on the event $\mathbb{Q}_n(m; t_1 \prec t_2)$, i.e., the event that m out of n received t_1 before t_2 , can be determined by applying Bayes' theorem:

$$\Pr \left[\omega \mid \mathcal{E}_{m/n}^{t_1 \prec t_2} \right] = \frac{\Pr \left[\mathcal{E}_{m/n}^{t_1 \prec t_2} \mid \omega \right] \cdot \Pr [\omega]}{\int_{-\infty}^\infty \left(\Pr \left[\mathcal{E}_{m/n}^{t_1 \prec t_2} \mid \hat{\omega} \right] \cdot \Pr [\hat{\omega}] \right) d\hat{\omega}} \quad (15)$$

Altogether, it holds:

$$\begin{aligned} \mathbb{P}_n(m) &= \Pr \left[t_1 \ll t_2 \mid \mathcal{E}_{m/n}^{t_1 \prec t_2} \right] \\ &= \int_0^\infty \left(\frac{\Pr \left[\mathcal{E}_{m/n}^{t_1 \prec t_2} \mid \omega \right] \cdot \Pr [\omega]}{\int_{-\infty}^\infty \Pr \left[\mathcal{E}_{m/n}^{t_1 \prec t_2} \mid \hat{\omega} \right] \cdot \Pr [\hat{\omega}] d\hat{\omega}} \right) d\omega \end{aligned} \quad (16)$$

□

PROOF OF THEOREM 3. It holds for any observer o :

$$\bigwedge_i t_i \prec_o t \Leftrightarrow \tau_o(t) > \tau_C \quad \text{and} \quad \Pr [\tau_o(t) > \tau_C] = 1 - F_{\tau_o(t)}(\tau_C) \quad (17)$$

Then, assuming that $\tau_{o_i}(t_x)$ is independent from $\tau_{o_j}(t_x)$ for any $i \neq j$, it holds:

$$\Pr \left[\bigwedge_{o \in O} \left(\bigwedge_i t_i \prec_o t \right) \right] = \Pr [\tau_o(t) > \tau_C]^n = \left(1 - F_{\tau_o(t)}(\tau_C) \right)^n \quad (18)$$

□

D Observers located in US & Europe

This section reports selected results (Table 8) on transactions recorded in the Bitcoin and the Ethereum networks using observers located exclusively in the US and Europe (cf. Table 7), mimicking the current geographic distribution of Bitcoin and Ethereum nodes and Mempool.space [3] observers. Specifically, we report additional evaluation results using a subset of the transactions evaluated in Section 4 filtered by reception entries of nodes located in the US and Europe. This additional evaluation is motivated by the fact that, while our main evaluation builds on measurements recorded by observers globally distributed around the world (cf. Table 6), some blockchains have a heavy accumulation of nodes in the US

US & Europe only						
m	Bitcoin (3 134 824 Txs)			Ethereum (6 552 228 Txs)		
	Received Txs	$C_{order}(m)$ All Txs	FR Txs	Received Txs	$C_{order}(m)$ All Txs	FR Txs
≥ 6 nodes	99.44%	98.89%	100.0%	96.80%	93.48%	100.00%
≥ 7 nodes	99.38%	98.76%	100.0%	96.55%	92.68%	99.80%
≥ 8 nodes	99.25%	98.51%	100.0%	96.20%	91.56%	99.39%
≥ 9 nodes	99.11%	98.21%	100.0%	95.95%	90.71%	99.00%
≥ 10 nodes	98.81%	97.60%	100.0%	95.65%	89.88%	98.60%
≥ 11 nodes	97.99%	95.95%	99.99%	95.24%	88.90%	98.21%
≥ 12 nodes	95.04%	90.32%	99.98%	87.72%	75.27%	97.80%

Table 8: Received Txs denotes the number of transactions recorded by at least m out of $n = 16$ observers in Bitcoin and Ethereum. $C_{order}(m)$ denotes the fraction of transactions received in a consistent order by our 16 nodes. Results labeled with **FR Txs** are based on the intersection of transactions received by all 16 observers. Results labeled with **All Txs** include all transactions.

and Europe. That is, according to data from bitnodes.io, ethernodes.org, and mempool.space, the majority of both Bitcoin's and Ethereum's nodes are located in these regions. Nevertheless, our findings reported in Section 4.1 and Section 4.2 remain consistent regardless of whether observers are restricted to the US and Europe or distributed more globally, including regions such as Singapore.

E Impact of Measurement Time

In addition to the empirical result reported in Section 4, we provide additional results from an initial measurement with 20 Bitcoin nodes captured in October 2024. The locations of our 20 nodes are highlighted in Table 9.

Our observers were similarly hosted by Hetzner cloud [21] with an identical setup, i.e., four dedicated vCPU kernels, 16GB RAM, SSD storage, an AMD EPYC-Milan processor, and reachable by an IPv4 address. For software, we used *bitcoind 25.0* as our Bitcoin client. Identically to the evaluation in Sections 4.1, each observer logged all incoming transactions and their exact reception time, and we compare the transactions' reception times among our observers.

State/Country	Network zone	#Nodes
Virginia, USA	us-east	4
Oregon, USA	us-west	4
Germany (south)	eu-central	4
Germany (east)	eu-central	4
Finland	eu-north	4
		20

Table 9: The locations of the 20 nodes from our initial measurement.

Over a period of 189 hours, we recorded approximately 5.7 million Bitcoin transactions and 3.4 million Ethereum transactions over 106 hours. Our results on the Bitcoin measurement is highlighted in Table 10. It shows that only 94.2% of all Bitcoin transactions were received by all observers, while 98.21% reached at least 15, and 98.99% reached a majority of at least 11 nodes. On average, each node received $98.5\% \pm 0.33\%$ of all transactions. We again determine an empirical estimate of $\mathbb{Q}_n(m)$ by sampling 100 000 random pairs of transactions $(t_0^{(i)}, t_1^{(i)})$ with $i \in I := [1, 100\,000]$, i.e., 200 000

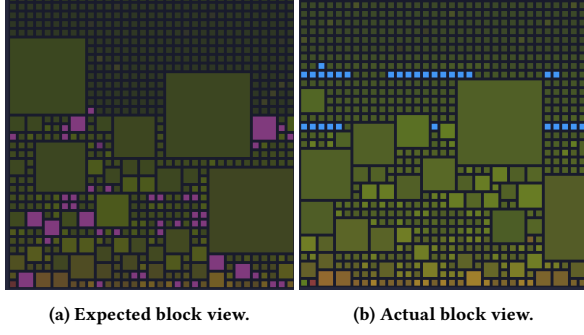


Figure 8: An excerpt of the block audit by the Mempool Open Source Project [3] for Bitcoin block 859 502. This example shows the comparison of (a) the expected block and (b) the actual block. Each square denotes a transaction, with the square’s color indicating the transaction’s label.

m	Received Txs		$C_{order}(m)$	
	Abs.	Rel.	All Txs	FR Txs
≥ 10 nodes	5 658 000	99.08%	98.12%	100.0%
≥ 11 nodes	5 652 775	98.99%	97.94%	100.0%
≥ 12 nodes	5 647 283	98.98%	97.73%	99.99%
≥ 13 nodes	5 638 991	98.75%	97.43%	99.99%
≥ 14 nodes	5 625 313	98.51%	96.96%	99.98%
≥ 15 nodes	5 607 949	98.21%	96.36%	99.97%
≥ 18 nodes	5 521 894	96.70%	93.40%	99.91%
20 nodes	5 380 042	94.21%	88.51%	99.72%
Total	5 710 445			

Table 10: *Received Txs* denotes the number of transactions recorded by at least m out of $n = 20$ observers in the Bitcoin network. $C_{order}(m)$ denotes the fraction of transactions received in a consistent order by our 20 nodes. Results labeled with *FR Txs* are based on the intersection of transactions received by all 20 observers. Results labeled with *All Txs* include all transactions.

transactions uniform at random from our recorded transaction sets and pair them up randomly. We observe that 88.51% of the randomly drawn transaction pairs from the complete dataset are witnessed by all 20 observers in the same order. For transactions that are received by all observers (*FR Txs*), we observe that 99.4% of the sampled pairs were received in consistent order.

The results in Table 10 show similar trends to those in Table 4, indicating that our findings are robust across different measurement periods.

F Block Audit

The Mempool Open Source Project [3] provides a preview of a potential block template. This block template incorporates all transactions that are expected to be included in the next block. Once a block is mined, the tool compares the expected block template with the actual block and highlights all differences (cf. Figure 8). In particular, it assesses each transaction according to its reception time and fees. For instance, the tool exposes transactions added to the block template without being received by any of their nodes and transactions with marginal fees. Figure 8 depicts a block audit for Bitcoin block 859 502. Here, blue-colored transactions in Figure 8b denote transactions that are present in the actual block but not included in the expected block. Pink-colored transactions in Figure 8a denote transactions that are present in the expected block but not in the actual block.